BROWSERMIND DATA

Especificación funcional inicial del MVP

Versión 3 — 24 de octubre de 2026

Estado

Documento de propuesta y especificación inicial. No describe funcionalidades ya implementadas.

Si el proyecto es aceptado en el hackathon, este documento será la base de implementación y se ajustará a las reglas oficiales publicadas por Meta y Cerebral Valley.

Objetivo

Construir un agente privado de diagnóstico y reparación para notebooks y scripts de data science en Python.

BrowserMind debe intentar resolver problemas localmente mediante reglas, análisis estructural y verificadores. Sólo cuando sea necesario debe pedir a un modelo de Meta que genere código. El sistema debe enviar únicamente el contexto necesario: celdas, funciones, traceback y resultados del checker.

La visión futura es soportar otros lenguajes y tareas de programación mediante adaptadores. El MVP se limita deliberadamente a Python para data science.

Casos De Uso Del Mvp

1. Detectar y corregir KeyError por nombre de columna.
2. Corregir problemas de tipos en pandas y NumPy.
3. Corregir shape mismatch en scikit-learn.
4. Detectar y corregir NaN o valores inválidos antes de entrenar un modelo.
5. Reparar errores de merge, indexación o alineación de DataFrames.
6. Corregir una celda o función y verificar que el resultado esperado se mantenga.

No se promete resolver cualquier problema de Python, entrenar modelos grandes ni soportar todas las librerías científicas.

Flujo Principal

1. El usuario carga un notebook .ipynb, un archivo .py o un ejemplo público.
2. BrowserMind identifica celdas, imports, variables, funciones y dependencias.
3. El usuario describe el problema o pega el traceback.
4. El sistema clasifica el error.
5. El sistema busca las celdas y símbolos relacionados.
6. El sistema intenta una solución determinista mediante reglas o plantillas.

7. Si no alcanza, solicita una acción pequeña a un modelo de Meta.
8. Si hace falta escribir código, envía sólo el contexto seleccionado.
9. Pyodide ejecuta el código en el navegador.
10. Un checker verifica el resultado.
11. Si falla, se envían sólo el error, el diff y el contexto adicional necesario.
12. El sistema escala a un modelo más potente cuando se agota el presupuesto o la confianza es baja.

Arquitectura

Interfaz y orquestador en el navegador:

- React + TypeScript.
- Parser de notebooks .ipynb.
- Índice de celdas, variables, imports y dependencias.
- Clasificador de traceback.
- Reglas y plantillas deterministas.
- Selector progresivo de contexto.
- Presupuesto de llamadas y tokens generativos.
- Memoria de sesión en IndexedDB.
- Registro visible de decisiones, contexto enviado y resultados.

Ejecución local:

- Pyodide/WebAssembly.
- NumPy, pandas, SciPy, Matplotlib y scikit-learn sólo cuando estén disponibles y verificadas en el entorno.
- Ejecución dentro de un Web Worker para no bloquear la interfaz.

Modelos de Meta:

- Nivel 0: reglas y plantillas, sin llamada generativa.
- Nivel 1: modelo pequeño de Meta, opcional y sólo si funciona de forma estable.
- Nivel 2: Muse Glimmer como generador principal de parches.
- Nivel 3: Muse Spark como escalamiento y referencia de alta capacidad.

La infraestructura puede usar runtimes o endpoints externos, pero el modelo principal y la referencia experimental deben pertenecer al ecosistema de Meta. No se requiere Gemma ni LiteRT-LM para el MVP.

Privacidad

Modo privado:

- El dataset permanece en el navegador.
- No se envía el dataset completo al modelo.
- Se filtran rutas, valores sensibles, filas de ejemplo y datos personales del traceback.
- El modelo recibe sólo código, error y contexto mínimo.

Modo benchmark Meta:

- Se utilizan datos sintéticos, públicos o anonimizados.
- Glimmer y Spark pueden ejecutarse mediante endpoints remotos.
- Este modo minimiza la exposición del código, pero no debe presentarse como privacidad total.

Verificación

Cada tarea del benchmark tendrá un checker explícito. No se considera resuelta sólo porque desaparezca el traceback.

Los checkers pueden verificar:

- nombres y tipos de columnas;
- shape de matrices;
- ausencia de NaN;
- valores esperados;
- rango mínimo de una métrica;
- igualdad aproximada de resultados;
- ejecución completa de una función o celda;
- ausencia de regresiones conocidas.

Benchmark

El benchmark inicial tendrá entre 15 y 30 tareas de Python data science.

Cada tarea incluye:

- notebook o script;
- error reproducible;
- datos sintéticos o autorizados;
- checker independiente;
- resultado esperado;
- registro de si la solución fue correcta.

Configuraciones:

A. Modelo Meta con contexto convencional.

B. Modelo Meta con recuperación simple por fragmentos.

C. BrowserMind con reglas, selección estructural y escalamiento.

D. Muse Spark con las mismas herramientas como referencia.

Métricas:

- porcentaje de tareas resueltas;
- porcentaje resuelto sin generación;
- tokens generativos por tarea resuelta;

- número de llamadas al modelo;
- celdas y símbolos enviados;
- latencia;
- costo estimado;
- regresiones;
- porcentaje de soluciones aceptadas por el checker.

Objetivo Experimental

Medir si BrowserMind mantiene una tasa de resolución comparable a una estrategia RAG convencional usando menos tokens generativos y enviando menos código.

Los resultados son objetivos experimentales, no promesas. No se afirmará paridad general con Muse ni aprendizaje de parámetros si sólo existe recuperación y modificación de código.

Criterios De Aceptación Del Mvp

El MVP se considera funcional si:

1. Carga y ejecuta al menos un notebook o script Python en el navegador.
2. Reproduce al menos cinco errores de data science.
3. Corrige automáticamente al menos una clase de error.
4. Ejecuta un checker después del parche.
5. Muestra qué contexto se envió al modelo.
6. Registra tokens, llamadas y tiempo.
7. Puede escalar desde reglas a Glimmer y, opcionalmente, a Spark.
8. Tiene una demo reproducible con datos públicos o sintéticos.

Plan De Implementación

Día 1: interfaz, carga de notebook y ejecución básica con Pyodide.

Día 2: parser de celdas, imports y variables.

Día 3: clasificación de traceback y reglas locales.

Día 4: primer checker y primera reparación automática.

Día 5: integración con Glimmer y contexto selectivo.

Día 6: reintentos con diff y traceback; límites de tokens.

Día 7: integración opcional con Spark y registro de métricas.

Día 8: benchmark y datos sintéticos.

Día 9: interfaz de observabilidad, video y modo replay.

Día 10: correcciones, documentación y entrega.

Fuera Del Mvp

- Soporte universal para todos los lenguajes.
- Entrenamiento de un LLM desde cero.
- Reconstrucción o ingeniería inversa de los pesos de Muse.
- Soporte garantizado para PyTorch, TensorFlow, XGBoost o cualquier paquete no verificado en Pyodide.
- Promesa de privacidad cuando se usan endpoints remotos.
- Escritura directa sobre la rama principal de un repositorio.

Visión

BrowserMind evolucionará hacia un núcleo de orquestación independiente del lenguaje. Cada adaptador futuro implementará:

- index();
- locate(error);
- select_context();
- apply_patch();
- verify();

El primer adaptador será Python data science. Luego podrán agregarse TypeScript, JavaScript, SQL y otros entornos.

Pitch Corto

BrowserMind is a privacy-first data-science debugging agent. It resolves structured Python errors locally whenever possible and escalates to Meta models only when code generation is necessary. The browser indexes notebook cells, selects only the relevant context, executes Python through WebAssembly, and verifies every patch with an independent checker. The project measures generative tokens per resolved error, code exposure, model calls and success rate across deterministic rules, Meta open-weight inference and Muse Spark.